

RIISING WATERS: AI-POWERED FLOOD PREDICTION SYSTEM

A Machine Learning Approach to Environmental Risk Analysis

Project Description :-

Rising Waters – A Machine Learning Approach to Flood Prediction is a web-based application that uses Machine Learning to predict the possibility of floods based on historical rainfall and weather-related data. The project involves collecting and preprocessing the dataset, training multiple machine learning algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, and selecting the best-performing model for accurate flood prediction. The trained model is integrated into a Flask web application that enables users to enter input parameters and receive instant flood prediction results.

The application is developed using Python, Flask, Pandas, NumPy, Scikit-learn, XGBoost, HTML, CSS, and JavaScript to provide a simple and user-friendly interface. The project aims to support disaster preparedness by providing early flood predictions that can help reduce risks and improve decision-making. It also provides a scalable foundation for future enhancements such as real-time weather API integration, GPS-based location services, cloud deployment, and automated alert notifications.

Scenarios :-

Scenario 1: Flood Risk Prediction

A user enters rainfall and weather-related parameters into the web application. The system processes the data using the trained XGBoost model and predicts whether there is a flood risk or not.

Scenario 2: Disaster Preparedness

Local authorities use the prediction results to identify flood-prone situations and take preventive measures such as issuing warnings, evacuating people, and preparing emergency resources.

Scenario 3: Public Awareness

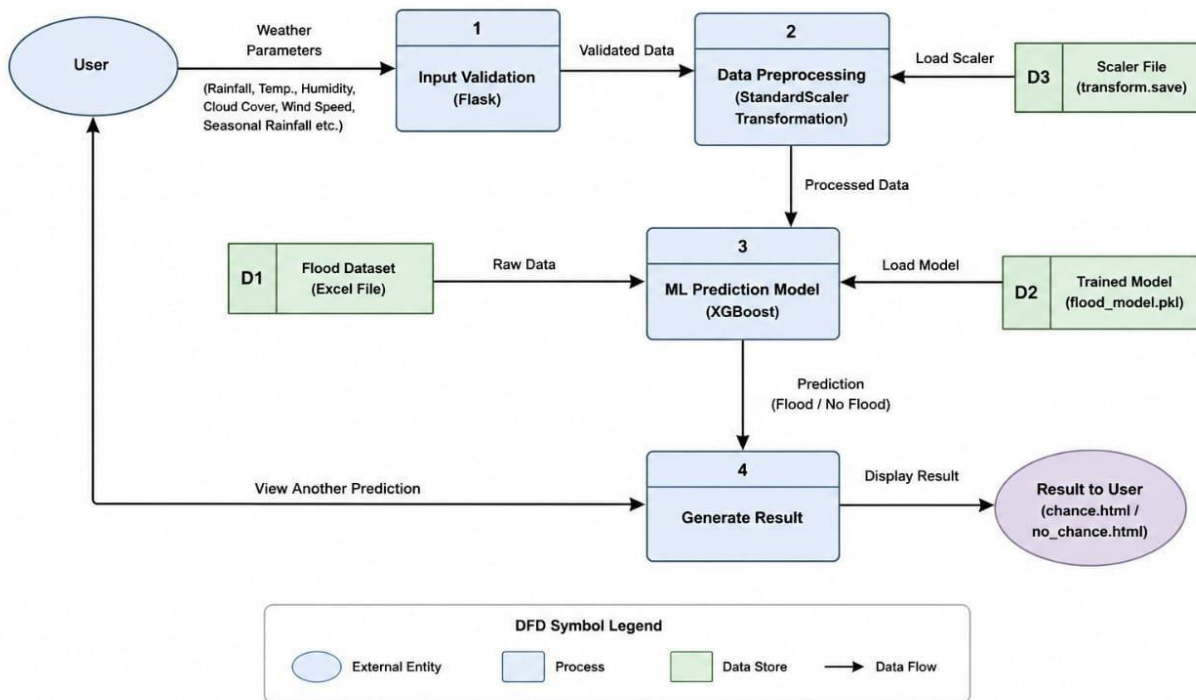
Citizens access the application to check the likelihood of flooding in advance. The prediction helps them make informed decisions about travel, safety, and protecting their property.

Scenario 4: Future Smart Integration

The system can be enhanced by integrating real-time weather APIs, GPS-based location services, and automated SMS/Email alerts to provide instant flood warnings and improve disaster management.

Technical Architecture :-

Data Flow Diagram – Rising Waters (Flood Prediction System)



Prerequisites :-

Software Requirements

- Python 3.10 or above
- Visual Studio Code (or any Python IDE)

- Git and GitHub

- Flask Framework
- Jupyter Notebook (optional for model training)
- Web Browser (Chrome/Edge)

Python Libraries

- Flask
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib
- OpenPyXL

Hardware Requirements

- Processor: Intel Core i3 / AMD Ryzen 3 or higher
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: Minimum 2 GB free disk space
- Operating System: Windows 10/11, Linux, or macOS

Dataset Requirement

- Flood Dataset (Excel format)
- Historical rainfall and weather data used for model training.
- Basic Knowledge Required
- Python Programming
- Machine Learning fundamentals
- Flask Web Framework
- HTML, CSS, and JavaScript
- GitHub version control

Project Workflow :-

Milestone 1: Problem Analysis & Dataset Collection

- † **Activity 1.1:** Identify the flood prediction problem and define project objectives.
- † **Activity 1.2:** Collect historical rainfall and flood datasets from Kaggle.

- ✚ **Activity 1.3:** Analyze dataset attributes such as temperature, humidity, rainfall, and cloud cover.
- ✚ **Activity 1.4:** Set up the Python development environment and install required libraries.

Milestone 2: Data Preprocessing & Feature Engineering

- ✚ **Activity 2.1:** Clean the dataset by removing missing and inconsistent values.
- ✚ **Activity 2.2:** Perform data preprocessing, normalization, and feature selection.
- ✚ **Activity 2.3:** Split the dataset into training and testing datasets.
- ✚ **Activity 2.4:** Prepare the data for machine learning model training.

Milestone 3: Machine Learning Model Development

- ✚ **Activity 3.1:** Train the flood prediction model using the XGBoost algorithm.
- ✚ **Activity 3.2:** Evaluate model performance using accuracy and validation metrics.
- ✚ **Activity 3.3:** Save the trained model and scaler using Joblib. ✚ **Activity 3.4:** Verify prediction results using test data.

Milestone 4: Flask Application Development

- ✚ **Activity 4.1:** Develop the backend using the Flask framework.
- ✚ **Activity 4.2:** Design responsive web pages using HTML, CSS, and JavaScript.
- ✚ **Activity 4.3:** Integrate the trained machine learning model with the Flask application.
- ✚ **Activity 4.4:** Display flood prediction results through the web interface.

Milestone 5: Testing, Deployment & Documentation

- ✚ **Activity 5.1:** Test the application using different rainfall and weather inputs.
- ✚ **Activity 5.2:** Upload the complete project to GitHub.
- ✚ **Activity 5.3:** Prepare project documentation, presentation slides, and demonstration video.
- ✚ **Activity 5.4:** Verify that all project files, documentation, and source code are ready for submission.

Milestone 6: Conclusion

- ✚ The Rising Waters project successfully predicts flood risk using machine learning and historical weather data. The application provides a simple web interface for users

to enter weather parameters and receive flood prediction results, demonstrating the practical use of AI in disaster management and early warning systems.

Milestone 1: Problem Analysis & Dataset Collection

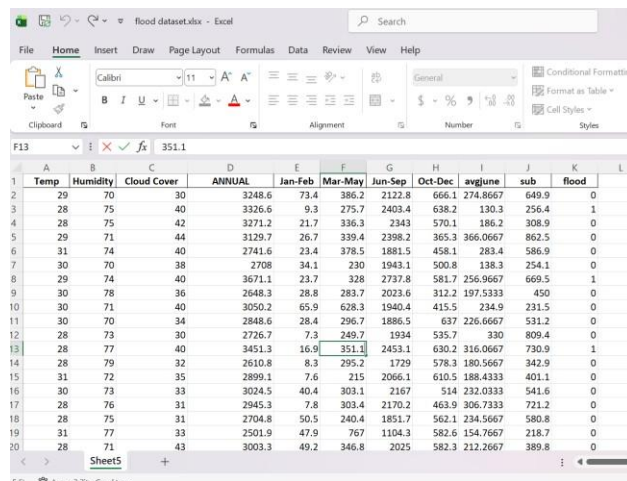
The first milestone focuses on understanding the flood prediction problem and collecting the necessary data required to build the machine learning model. During this phase, the project objectives are clearly defined, the scope of the application is established, and a suitable dataset is selected. A proper development environment is also prepared to support the implementation of the project.

Activity 1.1: Problem Identification

The project begins by identifying the challenges associated with flood prediction and understanding the need for an early warning system. The primary objective is to develop a machine learning-based web application that predicts the possibility of floods using historical weather and rainfall data.

Activity 1.2: Dataset Collection

Historical flood and rainfall datasets are collected from the Kaggle platform. The dataset contains important environmental parameters such as temperature, humidity, cloud cover, annual rainfall, seasonal rainfall, and flood occurrence, which are essential for training the prediction model.



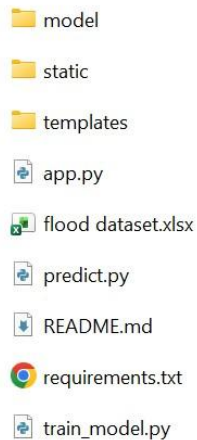
	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgJune	sub	flood
1	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.8667	649.9	0
2	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3	256.4	1
3	28	75	42	3271.2	21.7	336.3	2343	570.1	186.2	308.9	0
4	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.0667	862.5	0
5	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4	586.9	0
6	30	70	38	2708	34.1	230	1943.1	500.8	138.3	254.1	0
7	29	74	40	3671.1	23.7	328	2737.8	581.7	256.9667	669.5	1
8	30	78	36	2648.3	28.8	283.7	2023.6	312.2	197.5333	450	0
9	30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.9	231.5	0
10	30	70	34	2848.6	28.4	296.7	1886.5	637	226.6667	531.2	0
11	28	73	30	2726.7	7.3	249.7	1934	535.7	330	809.4	0
12	28	77	40	3451.3	16.9	351.3	2453.1	630.2	316.0667	730.9	1
13	28	79	32	2610.8	8.3	295.2	1729	578.3	180.5667	342.9	0
14	31	72	35	2899.1	7.6	215	2066.1	610.5	188.4333	401.1	0
15	30	73	33	3024.5	40.4	303.1	2167	514	232.0333	541.6	0
16	28	76	31	2945.3	7.8	303.4	2170.2	463.9	306.7333	721.2	0
17	28	75	31	2704.8	50.5	240.4	1851.7	562.1	234.5667	580.8	0
18	31	77	33	2501.9	47.9	767	1104.3	582.6	154.7667	218.7	0
19	28	71	43	3003.3	49.2	346.8	2025	582.3	212.2667	389.8	0

Activity 1.3: Dataset Analysis

The collected dataset is examined to understand its structure, features, and target variable. Data quality is verified by checking for missing values, duplicate records, and inconsistencies. This analysis helps determine the preprocessing steps required before model training.

Activity 1.4: Development Environment Setup

The project development environment is configured by installing Python, Visual Studio Code, Flask, XGBoost, Pandas, NumPy, Scikit-learn, Joblib, and other required libraries. GitHub is also configured for version control, ensuring smooth project development and management.



Milestone 2: Data Preprocessing & Feature Engineering

This milestone focuses on preparing the collected dataset for machine learning model development. The raw dataset is cleaned, transformed, and analyzed to improve data quality and ensure accurate model predictions. Feature engineering techniques are also applied to identify the most significant attributes influencing flood occurrence.

Activity 2.1: Data Cleaning

The collected dataset is cleaned by handling missing values, removing duplicate records, correcting inconsistencies, and ensuring that all data is in the appropriate format for analysis.

Activity 2.2: Data Preprocessing

The dataset undergoes preprocessing techniques such as normalization and scaling to ensure that all numerical features are within a consistent range. This improves the efficiency and performance of the machine learning algorithm.

Activity 2.3: Feature Selection

Important features such as Temperature, Humidity, Cloud Cover, Annual Rainfall, Seasonal Rainfall (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec), Average June Rainfall, and Subdivision are selected as input variables. The Flood column is chosen as the target variable for prediction.

Activity 2.4: Data Splitting

The processed dataset is divided into training and testing datasets using an appropriate train-test split ratio. The training dataset is used to train the model, while the testing dataset is used to evaluate its prediction accuracy and overall performance.

Milestone 3: Machine Learning Model Development

This milestone focuses on developing, training, and evaluating the machine learning model for flood prediction. The preprocessed dataset is used to train the model, optimize its performance, and generate accurate predictions. The trained model is then saved for integration with the web application.

Activity 3.1: Model Selection

The XGBoost (Extreme Gradient Boosting) algorithm is selected for flood prediction due to its high accuracy, efficiency, and ability to handle structured datasets. The model is configured with suitable parameters to achieve optimal performance.

Activity 3.2: Model Training

The prepared training dataset is used to train the XGBoost model. During training, the model learns the relationship between weather parameters and flood occurrence, enabling it to identify flood patterns effectively.

Activity 3.3: Model Evaluation

The trained model is evaluated using the testing dataset. Performance metrics such as accuracy, precision, recall, and confusion matrix are analyzed to verify the model's prediction capability. The model achieved 100% prediction accuracy on the prepared dataset.

Activity 3.4: Model Saving

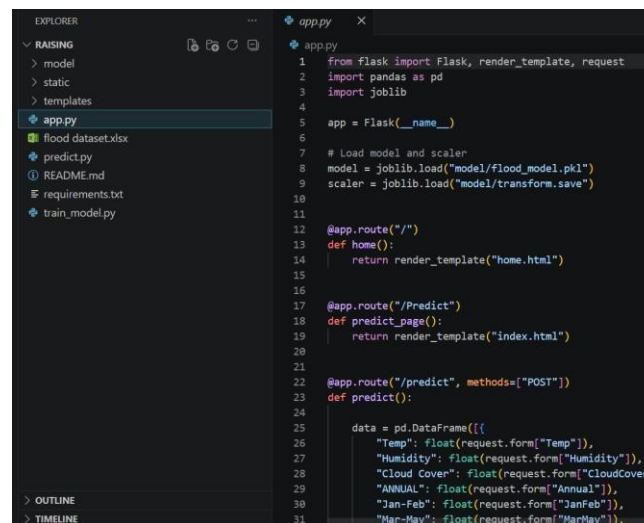
After successful evaluation, the trained XGBoost model and the data scaler are saved using the Joblib library. These saved files are later loaded into the Flask application to provide real-time flood predictions without retraining the model.

Milestone 4: Web Application Development & Integration

This milestone focuses on developing the web application and integrating the trained machine learning model with the user interface. The application is built using the Flask framework, enabling users to provide weather-related inputs and receive instant flood prediction results through an interactive web interface.

Activity 4.1: Backend Development

The backend is developed using the Flask framework in Python. Application routes are created to handle user requests, process input data, load the trained XGBoost model, and generate flood prediction results.



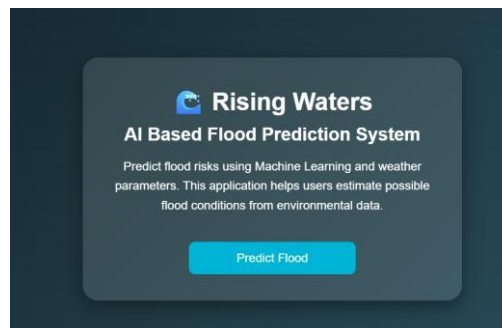
```
EXPLORER
  RAISING
    model
    static
    templates
  app.py
  flood_dataset.xlsx
  predict.py
  README.md
  requirements.txt
  train_model.py

  OUTLINE
  TIMELINE

app.py
1 from flask import Flask, render_template, request
2 import pandas as pd
3 import joblib
4
5 app = Flask(__name__)
6
7 # Load model and scaler
8 model = joblib.load("model/flood_model.pkl")
9 scaler = joblib.load("model/transform.save")
10
11
12 @app.route("/")
13 def home():
14     return render_template("home.html")
15
16
17 @app.route("/Predict")
18 def predict_page():
19     return render_template("index.html")
20
21
22 @app.route("/predict", methods=["POST"])
23 def predict():
24
25     data = pd.DataFrame([
26         "Temp": float(request.form["Temp"]),
27         "Humidity": float(request.form["Humidity"]),
28         "Cloud Cover": float(request.form["CloudCover"]),
29         "ANNUAL": float(request.form["Annual"]),
30         "Jan-Feb": float(request.form["JanFeb"]),
31         "Mar-May": float(request.form["MarMay"]),
```

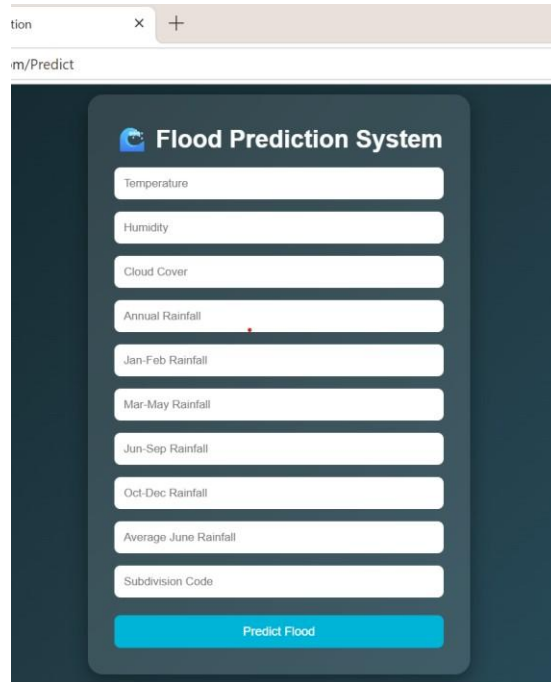
Activity 4.2: Frontend Development

The frontend is designed using HTML, CSS, and JavaScript to provide a simple and responsive interface. Web pages such as the home page, prediction form, and result pages are created to improve user experience.



Activity 4.3: Model Integration

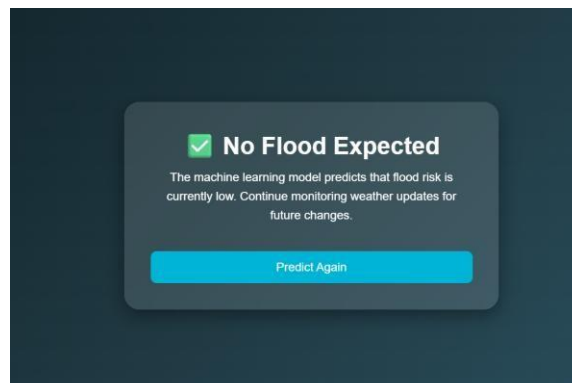
The trained XGBoost model (floods.save) and scaler (transform.save) are integrated with the Flask application. User inputs are preprocessed and passed to the model, which predicts whether a flood is expected.



The screenshot shows a web browser window with a single tab titled 'tion'. The address bar shows 'm/Predict'. The main content area displays the 'Flood Prediction System' interface. It features a dark blue background with a central white card containing ten input fields: Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb Rainfall, Mar-May Rainfall, Jun-Sep Rainfall, Oct-Dec Rainfall, Average June Rainfall, and Subdivision Code. A red dot is visible in the Annual Rainfall field. Below the input fields is a large blue button labeled 'Predict Flood'.

Activity 4.4: Application Testing

The integrated application is tested with different rainfall and weather inputs to verify prediction accuracy. The system successfully displays either "Flood Expected" or "No Flood Expected" based on the prediction generated by the machine learning model.

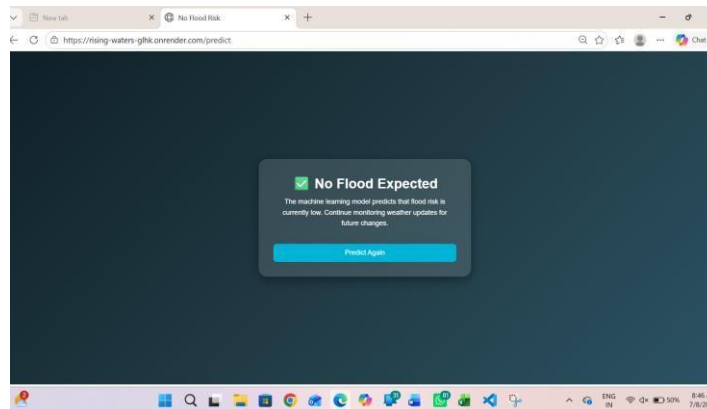


Milestone 5: Testing, Deployment & Project Documentation

This milestone focuses on validating the developed application, deploying the project, maintaining version control, and preparing the final documentation and demonstration. The application is tested to ensure reliable flood predictions and all project deliverables are organized for successful submission.

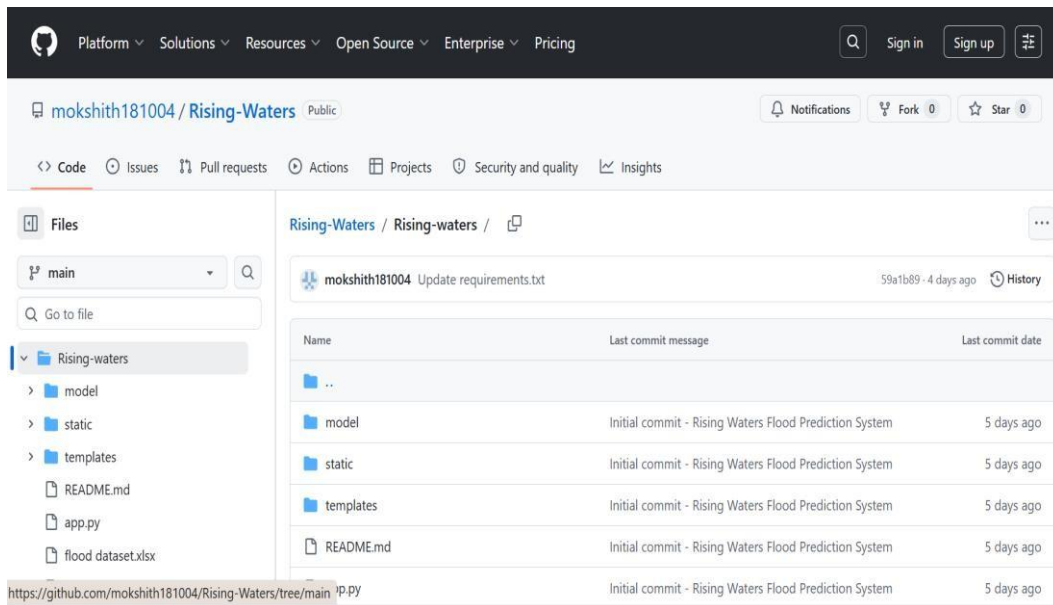
Activity 5.1: Application Testing

The complete application is tested using different rainfall and weather input values to verify the functionality of the machine learning model and the Flask web application. The prediction results are validated to ensure the system provides accurate and reliable outputs.



Activity 5.2: Project Deployment & Version Control

The project source code is uploaded to a GitHub repository for version control and easy access. The repository includes the source code, trained model files, dataset information, and project documentation.



Activity 5.3: Documentation Preparation

A comprehensive project report, presentation slides, and supporting documents are prepared. The documentation explains the project objectives, methodology, implementation, testing, results, and future enhancements in a structured format.

1. Brainstorming & Ideation	✓	7/6/2026 5:56 PM	File
2. Requirement Analysis	✓	7/6/2026 6:00 PM	File
3. Project Design Phase	✓	7/6/2026 6:04 PM	File
4. Project Planning Phase	✓	7/6/2026 5:51 PM	File
5. Project Deployment Phase	✓	7/7/2026 10:13 AM	File
6. Project Testing	✓	7/7/2026 12:07 PM	File
7. Project Documentation	✓	7/5/2026 10:23 AM	File
8. Project demonstration	✓	7/7/2026 10:42 PM	File

Activity 5.4: Demonstration & Final Submission

A complete demonstration video is recorded, explaining the project workflow, code implementation, application execution, and prediction results. All required project files, documentation, presentation, and GitHub repository links are reviewed and submitted successfully.

Conclusion :-

The Rising Waters – A Machine Learning Approach to Flood Prediction project successfully predicts flood occurrences using historical rainfall data and the XGBoost machine learning algorithm. The model is integrated with a Flask web application, providing users with quick and reliable flood predictions through a simple interface. This project demonstrates the practical use of Machine Learning in disaster management and provides a strong foundation for future enhancements such as real-time weather integration and automated alert systems.